# AWS Data Engineer — Glue, Crawler & Redshift

Interview guide: design, tune & troubleshoot pipelines

Role context · Q&A · real scenarios · design patterns · performance/cost · troubleshooting

Written for someone preparing for a **Data Engineer** role. The focus is on what a DE actually *does* with these services — designing pipelines, modeling data, tuning performance and cost, and fixing production issues — plus the **interview questions** you'll be asked and strong answers. Uses one running example: **"ShopFast," a retailer building an analytics platform.**

---

## PART 0 — What a Data Engineer does with these services

A data engineer owns the pipeline that turns raw data into trustworthy, query-ready data:

1. **Ingest** raw data into **S3** (the data lake) — from databases, APIs, logs, files.
2. **Transform** it with **AWS Glue** (serverless Spark) — clean, dedupe, join, reshape, convert to **Parquet**, partition it.
3. **Catalog** it with a **Glue Crawler** → **Data Catalog** so it's queryable by SQL.
4. **Serve** it: load into **Redshift** for fast BI, or query in place with Athena/Spectrum.
5. **Orchestrate, monitor, and guarantee quality** — schedule, handle failures, check data, control cost.

Interviewers probe all five. Below, each service is covered as **concepts → Q&A → real scenarios → gotchas**.

---

## PART 1 — AWS GLUE (data-engineer lens)

### Concepts a DE must own

- **Serverless Spark ETL** billed per **DPU-second** (worker types `G.1X` / `G.2X`, N workers).
- **Job Bookmarks** — process only *new* data on each run (the key to incremental pipelines).
- **DynamicFrame** (Glue's schema-flexible frame; supports bookmarks, `ResolveChoice` for messy types) vs Spark **DataFrame** (standard SQL-style ops). Convert between them freely.
- **Triggers & Workflows** — schedule (cron), chain jobs+crawlers, or run on events.
- **Glue Data Quality** — rule-based checks (DQDL) that can fail a pipeline on bad data.
- **Partitioning** and the **small-files problem** (below).

### Interview Q&A

**Q: How do you make a Glue job incremental instead of reprocessing everything?** Enable **job bookmarks** (`--job-bookmark-option job-bookmark-enable`) and read/write through `DynamicFrame` APIs with a stable `transformation_ctx`. Glue tracks processed files (by path/ timestamp) and only handles new data. Switch writes to **append** into date partitions (`.../dt=YYYY-MM-DD/`), not overwrite, so history is preserved. Reset the bookmark for a backfill.

**Q: DynamicFrame vs DataFrame — when do you use which?** DynamicFrame for messy/semi-structured input and when you need bookmarks or `ResolveChoice` (a column that's sometimes int,

sometimes string). Convert to DataFrame (`.toDF()`) for rich Spark SQL transforms, then back (`.fromDF()`) to write with Glue features.

**Q: What's the "small files problem" and how do you fix it?** Thousands of tiny files kill query performance (huge overhead per file) and inflate cost. Fixes: **coalesce/repartition** before writing (`df.repartition(10)`), enable Glue's `groupFiles`/`groupSize` for input, run a **compaction** job to merge small Parquet files, and partition sensibly (not too granular).

**Q: How do you orchestrate a multi-step pipeline?** Small cases: Glue **Workflows/Triggers** (job → crawler → job). Real production: **Step Functions** or **Managed Airflow (MWAA)** for DAGs with retries, branching, alerting, and event triggers (EventBridge on new S3 object).

**Q: Glue vs Lambda vs EMR?** Lambda = short (<15 min), lightweight, event-driven transforms. **Glue** = serverless Spark for standard batch ETL + cataloging. **EMR** = full control for huge/custom/long-running Spark with special libraries.

**Q: How do you handle schema evolution?** Parquet + Glue Catalog tolerate added columns; use DynamicFrame `ResolveChoice`/`ApplyMapping` to normalize types; enable crawler "update the table definition" or manage schema explicitly. For strict control, define tables in DDL and validate incoming schema.

### Real scenario

*"ShopFast's clickstream lands as thousands of tiny JSON files every hour; the dashboard is slow."* → Glue job (bookmarked) reads the hour's new files, **repartitions** to a handful of files, converts to **Parquet partitioned by date/hour**, writes to curated. A compaction job merges older partitions. Result: 10–50× fewer files, much faster Athena/Redshift queries.

### Gotchas

- Forgetting bookmarks → reprocessing everything nightly (slow + expensive).
- Over-partitioning (e.g., by minute) → millions of tiny partitions, worse than none.
- `transformation_ctx` renamed → bookmark resets, full reprocess.

---

## PART 2 — GLUE CRAWLER & DATA CATALOG (data-engineer lens)

### Concepts

- **Crawler** scans S3, uses **classifiers** to detect format, infers schema, detects partitions, and writes a **table** to the **Data Catalog** (metadata only — no data moves).
- **Data Catalog** = a **Hive metastore** shared by Athena, Redshift Spectrum, EMR, Glue.
- **Partition management** is the crawler's most valuable/annoying job.

### Interview Q&A

**Q: What exactly does a crawler create?** A database → table with column names/types, the S3 location, the format (SerDe), and **partition** metadata inferred from folder names (`region=US/` → a

`region` partition).

**Q: Crawler vs defining tables manually — which and when?** Crawler for **unknown/changing/exploratory** schemas. For **stable production** schemas, prefer explicit `CREATE TABLE` + **partition projection** — no crawler runs (less cost/ latency), exact types (no mis-inference), and it's version-controlled (IaC-friendly).

**Q: How do new daily partitions get picked up without a crawler?** Three options: re-run the crawler on a schedule; `MSCK REPAIR TABLE` / `ALTER TABLE ADD PARTITION`; or best — **partition projection** (Athena/Glue computes partitions from a rule, zero maintenance).

**Q: What are classifiers?** Rules that identify data format. Built-ins cover Parquet/ORC/JSON/CSV/Avro; **custom classifiers** (Grok/regex/JSON-path) handle odd formats like custom log lines.

**Q: A crawler typed all your CSV columns as `string` — why, and fix?** CSV is untyped, so inference is a guess and often defaults to string. Fix: use a **custom classifier** with types, or (better) define the table explicitly with correct types, or convert to **Parquet** (self-describing) upstream so types are exact.

### Real scenario

"*Analysts complain a new day's data 'isn't there' in Athena.*" → The partition wasn't registered. Short-term: run the crawler or `ALTER TABLE ADD PARTITION`. Permanent: switch the table to **partition projection** so new `dt=` folders are queryable instantly.

### Gotchas

- Pointing a crawler at a prefix that **mixes formats/schemas** → weird/extra tables.
- Scheduling crawlers **too often** → unnecessary cost + catalog churn.
- Relying on inference for CSV types in production.

---

## PART 3 — AMAZON REDSHIFT (data-engineer lens)

### Concepts a DE must own

- **Columnar + MPP**: leader node plans; compute nodes (split into **slices**) run in parallel. **RA3** separates compute/storage (managed storage on S3); **Serverless** removes cluster sizing.
- **Distribution style** (`DISTKEY`, `DISTSTYLE ALL`, `EVEN`) — controls where rows live.
- **Sort keys** (`SORTKEY`) — control on-disk order → **zone maps** skip blocks on filters.
- `COPY` (parallel bulk load from S3) / `UNLOAD` (export to S3).
- **Spectrum** — query S3 via the Glue Catalog, join to Redshift tables (lakehouse).
- **Housekeeping**: `VACUUM` (reclaim space, re-sort) and `ANALYZE` (refresh stats).
- **WLM** (workload management), **concurrency scaling**, **materialized views**, **data sharing**.

### Interview Q&A

**Q: How do you choose a distribution style?** `DISTKEY` on the **most-common join column** so joined rows sit on the same slice (avoids network shuffle) — e.g., `customer_id` on the orders fact. `DISTSTYLE ALL` for **small dimension** tables (copied to every node → local joins). `DISTSTYLE EVEN` when there's no clear join key. Bad DIST choice → **data skew** (one slice overloaded).

**Q: What's a sort key and why does it matter?** It orders rows on disk; Redshift keeps min/max **zone maps** per block, so a filter on the sort key (usually a **date**) skips irrelevant blocks — massive speedup for time-range queries. **Compound** sort key (prefix order matters) vs **interleaved** (equal weight to several columns; heavier to maintain).

**Q: How do you load data efficiently?** `COPY` **from S3** — it loads across all slices in parallel. Never insert row-by-row. Best practices: split input into multiple files (≥ number of slices) so every slice works, use **Parquet/columnar** or compressed files, and load from the **same region**.

**Q: How do you do an UPSERT / handle updates (SCD)?** Redshift has no native MERGE historically → the **staging + delete + insert** pattern: `COPY` new data into a **staging table**, `DELETE` matching keys from target, `INSERT` from staging (all in a transaction). For history, use **SCD Type 2** (add effective/expiry dates + a current flag). (Newer Redshift supports `MERGE`.)

**Q: What are VACUUM and ANALYZE, and when do you run them?** `VACUUM` reclaims space from deleted rows and **re-sorts** data (restores sort-key benefit); `ANALYZE` updates table **statistics** so the planner picks good plans. Run after big loads/ deletes. RA3/auto features do much of this automatically now, but know the concept.

**Q: How does Redshift handle many concurrent queries? WLM** (workload management) defines queues with memory/slots; **Automatic WLM** manages this for you. **Concurrency Scaling** spins up transient clusters during spikes so queries don't queue. **Materialized views** pre-compute expensive aggregations.

**Q: When Spectrum vs loading into Redshift?** Load **hot, frequently-queried** data into Redshift for speed; leave **cold/huge/rarely- queried** data in S3 and query via **Spectrum**, joining across both. Saves storage/compute and avoids loading everything.

**Q: RA3 vs DC2 vs Serverless?** DC2 = local storage, small performance-sensitive sets. **RA3** = compute/storage separated, scale independently, managed storage (the modern default). **Serverless** = no cluster to manage, pay per usage (RPUs) — great for variable workloads.

**Q: How do you troubleshoot a slow Redshift query?** Check `EXPLAIN` / `SVL_QUERY_REPORT`, look for **DS_BCAST/DS_DIST** (redistribution = bad dist key), **large scans** (missing sort-key filter), **skew** (uneven slice rows), and outdated stats (run `ANALYZE`). Fix dist/sort keys, add predicates, or a materialized view.

### Real scenario

"*ShopFast's* `revenue by region` *dashboard got slow as orders grew to billions of rows.*" → Set `SORTKEY(order_date)` (dashboards filter by date), `DISTKEY(customer_id)` (joins to customers), make

`regions` `DISTSTYLE ALL`, add a **materialized view** for the daily aggregate, enable **concurrency scaling** for peak hours. Query time drops from minutes to seconds.

### Gotchas

- Wrong DISTKEY → **skew** or constant redistribution.
- Row-by-row `INSERT` instead of `COPY` → painfully slow loads.
- Forgetting `ANALYZE` / `VACUUM` after big changes → bad plans, bloated tables.
- Loading one giant file → only one slice works (no parallelism).

---

## PART 4 — DATA MODELING & PIPELINE DESIGN PATTERNS (DE core)

- **Medallion / zones:** `raw` (immutable bronze) → `curated/cleaned` (silver) → `modeled/aggregated` (gold). Reprocess from raw anytime.
- **Star schema:** central **fact** tables (orders, events) + **dimension** tables (customer, product, date). Facts get the DISTKEY on the common join; small dims go `DISTSTYLE ALL`.
- **Slowly Changing Dimensions (SCD):** Type 1 = overwrite; **Type 2** = keep history with effective/expiry dates + current flag (very common interview topic).
- **File format & compression: Parquet/ORC** (columnar) for analytics; Snappy compression; Avro for row-based streaming. Columnar + partitioning = less scanned = faster + cheaper.
- **Partitioning strategy:** partition by the column you filter on most (usually **date**); avoid over-partitioning (millions of tiny partitions).
- **Incremental / CDC:** bookmarks or watermark columns; capture changes from source DBs with **DMS**; apply via staging + upsert.
- **Idempotency:** re-running a job must not double-count — use overwrite-by-partition or dedupe keys.
- **Batch vs streaming:** batch = Glue/EMR; streaming = **Kinesis/MSK** → **Flink/Spark Streaming** → **S3/Redshift** for near-real-time.

---

## PART 5 — PERFORMANCE & COST OPTIMIZATION (a DE's daily job)

**Glue:** right-size workers; enable bookmarks (don't reprocess); fix small files; push down filters; avoid unnecessary shuffles; use `G.2X` for memory-heavy jobs; turn on job metrics.

**S3/Athena: Parquet + partitioning + compression** = scan less = pay less (Athena bills per TB scanned); compact small files; `SELECT` only needed columns.

**Redshift:** correct DIST/SORT keys; `COPY` in parallel; compression (`ENCODE AUTO`); materialized views for repeated aggregations; concurrency scaling for spikes; pause/resize or **Serverless** for variable load; **Spectrum** to keep cold data off the cluster.

**Cost mantra:** scan less data, run compute only when needed, store cold data cheaply in S3.

---

## PART 6 — PRODUCTION TROUBLESHOOTING (real issues a DE fixes)

| Symptom | Likely cause | Fix |
|---|---|---|
| Glue job reprocesses everything | bookmarks off / ctx renamed | enable bookmarks, stable `transformation_ctx`, append to partitions |
| Query slow, thousands of files | small-files problem | repartition/compact to fewer, larger Parquet files |
| New data "missing" in Athena | partition not registered | run crawler / `ADD PARTITION` / partition projection |
| Redshift query very slow | bad dist key (redistribution) or missing sort filter | fix DISTKEY/SORTKEY, add predicates, `ANALYZE` |
| One slice much bigger (skew) | skewed DISTKEY | choose a higher-cardinality/even key or `EVEN` |
| Load takes forever | row-by-row INSERT / one big file | use `COPY` with many split files |
| Costs spiking | full scans / no partitioning / idle cluster | Parquet+partitions, select fewer cols, pause/Serverless |
| Glue OOM (executor) | data skew / huge partitions / big shuffle | `G.2X`, repartition, filter earlier, avoid collect() |

## PART 7 — RAPID-FIRE Q&A (say these fast)

- **ETL vs ELT?** ETL transforms before loading (Glue→Redshift); ELT loads raw then transforms in the warehouse (e.g., dbt in Redshift). Modern lakes lean ELT.
- **Why Parquet over CSV/JSON?** columnar + compressed + typed → scan less, faster, cheaper.
- **Athena vs Redshift?** Athena = serverless ad-hoc SQL on S3; Redshift = provisioned/serverless warehouse for frequent, high-concurrency BI.
- **What's the Glue Data Catalog?** shared Hive metastore (db→table→schema+partitions) used by Athena, Spectrum, EMR, Glue.
- **How to guarantee data quality?** Glue Data Quality / Great Expectations rules that fail the pipeline before bad data reaches BI; freshness/null/uniqueness/range checks.
- **Fact vs dimension?** fact = measurable events (orders); dimension = descriptive context (customer, product, date).
- **Idempotent pipeline?** re-running yields the same result — overwrite-by-partition or dedupe.
- **DISTKEY vs SORTKEY?** DISTKEY = *where* rows live (join co-location); SORTKEY = *order* on disk (skip blocks on filters).
- **How to load Redshift fast?** `COPY` from many split Parquet files in the same region.
- **Handle late-arriving data?** partition by event date, reprocess the affected partition, use upsert.

## PART 8 — WHITEBOARD SCENARIO (practice out loud)

"*Design a pipeline: ShopFast gets daily order files in S3 and needs a revenue dashboard.*"

1. **Land** raw files in `s3://.../raw/orders/dt=YYYY-MM-DD/` (immutable).
2. **Transform** with a **bookmarked Glue job**: clean, dedupe, cast, write **Parquet** partitioned by `dt` to `s3://.../curated/orders/`.
3. **Catalog** with a scheduled crawler (or explicit table + projection) → `shopfast_db.orders`.
4. **Model**: build a **star schema** (fact_orders + dim_customer/product/date), either in Redshift via `COPY` + transforms, or in S3 with dbt/Glue.
5. **Serve**: dashboards on Redshift (DISTKEY `customer_id`, SORTKEY `order_date`, MV for daily aggregates); keep cold history in S3 via **Spectrum**.
6. **Orchestrate** with Step Functions/MWAA; **event-trigger** on new file (EventBridge).
7. **Quality**: Glue Data Quality checks (no nulls in order_id, amount ≥ 0, freshness) that **fail** the run on bad data.
8. **Monitor & cost**: CloudWatch alarms on job failures/latency; partitioning + Parquet to minimize scan cost; concurrency scaling for peak dashboard hours.

Mention **idempotency** (overwrite the day's partition on reruns) and **backfills** (reset bookmark, reprocess a date range) — interviewers love those.

---

## PART 9 — BEST-PRACTICES CHECKLIST

- [ ] Raw zone is **immutable**; transformations are **reproducible**.
- [ ] Store analytics data as **Parquet**, **partitioned by date**, compressed.
- [ ] Glue jobs are **incremental** (bookmarks) and **idempotent**.
- [ ] Avoid the **small-files** problem (repartition/compact).
- [ ] Catalog via crawler (exploratory) or **explicit DDL + partition projection** (prod).
- [ ] Redshift: right **DISTKEY/SORTKEY**, `COPY` loads, MVs, concurrency scaling.
- [ ] **Data quality** checks that fail the pipeline on bad data.
- [ ] **Orchestration** with retries/alerting (Step Functions/MWAA), event-driven where useful.
- [ ] **Least-privilege IAM**, encryption (SSE-KMS), and everything as **IaC** (Terraform/CFN).
- [ ] **Cost:** scan less (partitions/Parquet), compute only when needed, cold data in S3.